CO-1-ASSIGNMENT

Assignment title: **Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System**

Course Outcome	CO1
Assignment	Assignment tool 1
Faculty	Dr. F. Mary Harin Fernandez
Student Name	Pooja Sree M
Register Number	192521403
Course Code	CSA0608
Course Name	Design and Analysis of Algorithm

Abstract

This report compares three distinct strategies for searching a student registration number: iterative linear search, recursive binary search, and hash-table lookup. The assignment requires analysis for datasets of approximately 1,000, 10,000, 100,000 and 1,000,000 records, including mathematical complexity, best/average/worst cases, recurrence analysis, experimental measurements and the Day-4 requirement of 50,000 searches per hour. A Python benchmark script and measured output files accompany this report. Timings depend on the computer and Python runtime; rerun the supplied script on the submission computer and capture genuine screenshots before final submission.

Problem statement and assumptions

Each student has a unique registration number. The database must determine whether a requested number exists and return its record/index. The four tested dataset sizes follow the assignment. For a fair comparison, the benchmark creates the same unique integer keys, uses 30 successful random searches per algorithm and size, and checks missing-key cases separately. Search time is measured separately from preprocessing; preprocessing is nevertheless included in the theoretical total-cost discussion.

Task 1 – Algorithm design

Approach A: Iterative Linear Search

Design strategy: sequential scan; works on unsorted records and requires no preprocessing. Compare the target with each key from the beginning until a match is found or the list ends.

```
LINEAR-SEARCH(A, target)
  for i = 0 to length(A)-1
    if A[i] == target: return i
  return NOT_FOUND
```

Selected because it is simple, supports unsorted data and provides a baseline. It can be slow when the match is late or absent.

Approach B: Recursive Binary Search

Design strategy: divide and conquer; requires the records to be sorted by registration number. Compare the middle key and recursively search only the half that can contain the target.

```
RECURSIVE-BINARY-SEARCH(A, target, low, high)
  if low > high: return NOT_FOUND
  mid = floor((low + high)/2)
  if A[mid] == target: return mid
  if target < A[mid]: return RECURSIVE-BINARY-SEARCH(A,target,low,mid-1)
  else: return RECURSIVE-BINARY-SEARCH(A,target,mid+1,high)
```

Selected because halving the search interval gives logarithmic query work. Sorting is a preprocessing requirement and must be counted in the overall cost.

Approach C: Hash Table Search

Design strategy: hashing/indexing; build a dictionary mapping each registration number to its record position. A query hashes the key and looks up its entry. Expected lookup time is constant under a suitable hash distribution and controlled load factor; collisions can make worst-case lookup linear.

```
BUILD-HASH-TABLE(records)
  H = empty hash table
  for each record at index i: H[record.registration_number] = i
  return H
HASH-SEARCH(H, target)
  if target is in H: return H[target]
  return NOT_FOUND
```

Selected because the challenge has many searches but only one daily update batch. The index can be built or updated when records are added, then reused for many queries.

Task 2 – Basic operation

Algorithm	Basic operation	Why it dominates
Linear Search	Equality test A[i] == target	The scan repeats key comparisons until a match is found or the end is reached. The count grows with the position and input size \(n\).
Recursive Binary Search	Compare target with the middle key	Each comparison discards about half the remaining interval.

Hash Table	Hash computation and bucket/key lookup	Expected one indexed lookup; collision resolution may require extra key comparisons.

Task 3 – Mathematical complexity analysis

A. Linear search

Let n be the number of records. If the target is at zero-based position k, comparisons C(n)=k+1. If absent, C(n)=n. Thus best C(n)=1; worst C(n)=n. If a successful target is equally likely to occupy any position, average comparisons are (1+2+…+n)/n = [n(n+1)/2]/n = (n+1)/2. Therefore search time is Θ(n) on average and worst case, with best-case Θ(1). No preprocessing is needed, so total cost per search remains Θ(n).

B. Recursive binary search

For n>1, one middle comparison is made and only one subproblem of size approximately n/2 is searched. The recurrence is T(n)=T(⌊n/2⌋)+c, with T(0)=c0. After j steps, the interval is approximately n/2^j. Stop when n/2^j ≤ 1, giving j ≥ $\log_2$ n. Hence T(n)=Θ(log n). Best case is Θ(1) when the first midpoint matches; average and worst cases are Θ(log n). Sorting costs Θ(n log n) using a comparison sort. If the data are initially unsorted, one-time total cost for q searches is Θ(n log n + q log n); if already maintained in sorted order, a query costs Θ(log n), but inserts may require maintenance.

C. Hash-table search

Building the table takes Θ(n) expected time and Θ(n) additional space. Under a well-distributed hash function and controlled load factor, expected query time is Θ(1); the worst case is Θ(n) if many keys collide. For q queries, expected total cost after building is Θ(n+q), versus Θ(qn) for repeated linear search. In this implementation, a Python dictionary is used. The benchmark counts a dictionary lookup as one abstract lookup operation; its internal collision probes are not exposed by Python and therefore are not directly counted.

Task 4 – Best, average and worst cases

Algorithm	Best Case	Average Case	Worst Case	Input Condition
Linear Search	\(\Theta(1)\)	\(\Theta(n)\)	\(\Theta(n)\)	Best: first item matches. Average:

				target is equally likely to be at any position. Worst: target is last or absent.
Recursive Binary Search	\(\Theta(1)\)	\(\Theta(\log n)\)	\(\Theta(\log n)\)	Best: middle item matches immediately. Otherwise, the search interval is halved at each step until the target is found or the interval is empty.
Hash Table	Expected \(\Theta(1)\)	Expected \(\Theta(1)\)	\(\Theta(n)\)	Best/average: few collisions. Worst: many keys collide in the same bucket.

Task 5 – Asymptotic analysis

Linear search: best $\Omega(1)$, worst $O(n)$, average $\Theta(n)$. Recursive binary search: best $\Omega(1)$, worst $O(\log n)$, and average $\Theta(\log n)$, provided input is sorted. Hash lookup: expected $\Theta(1)$, worst $O(n)$, plus $\Theta(n)$ expected index construction. As n increases from 1,000 to 1,000,000, linear search may require roughly 1,000 times as many comparisons in a comparable worst-case query. Binary search adds only about $\log_2(1{,}000{,}000) \approx 20$ levels. Hash lookup is expected to remain near one indexed lookup per request, but consumes extra memory and relies on good hashing.

Task 6 – Recurrence formulation

For recursive binary search, the original input interval has n keys. Each non-base call makes one recursive call on at most half the interval and performs constant work (middle calculation and

comparisons) outside the recursive call. The base case occurs when low > high (empty interval) or a key matches. Therefore $T(n)=T(\lfloor n/2 \rfloor)+c$ for $n>1$, and $T(0)=\Theta(1)$. There is one recursive call per level, not two.

Task 7 – Recurrence solution by iteration

$T(n) = T(n/2) + c$

$= T(n/4) + 2c$

$= T(n/8) + 3c$

$= T(n/2^j) + jc$

Stop when $n/2^j <= 1$, so $j >= \log_2(n)$.

Therefore $T(n) = T(1) + c \log_2(n) = \Theta(\log n)$.

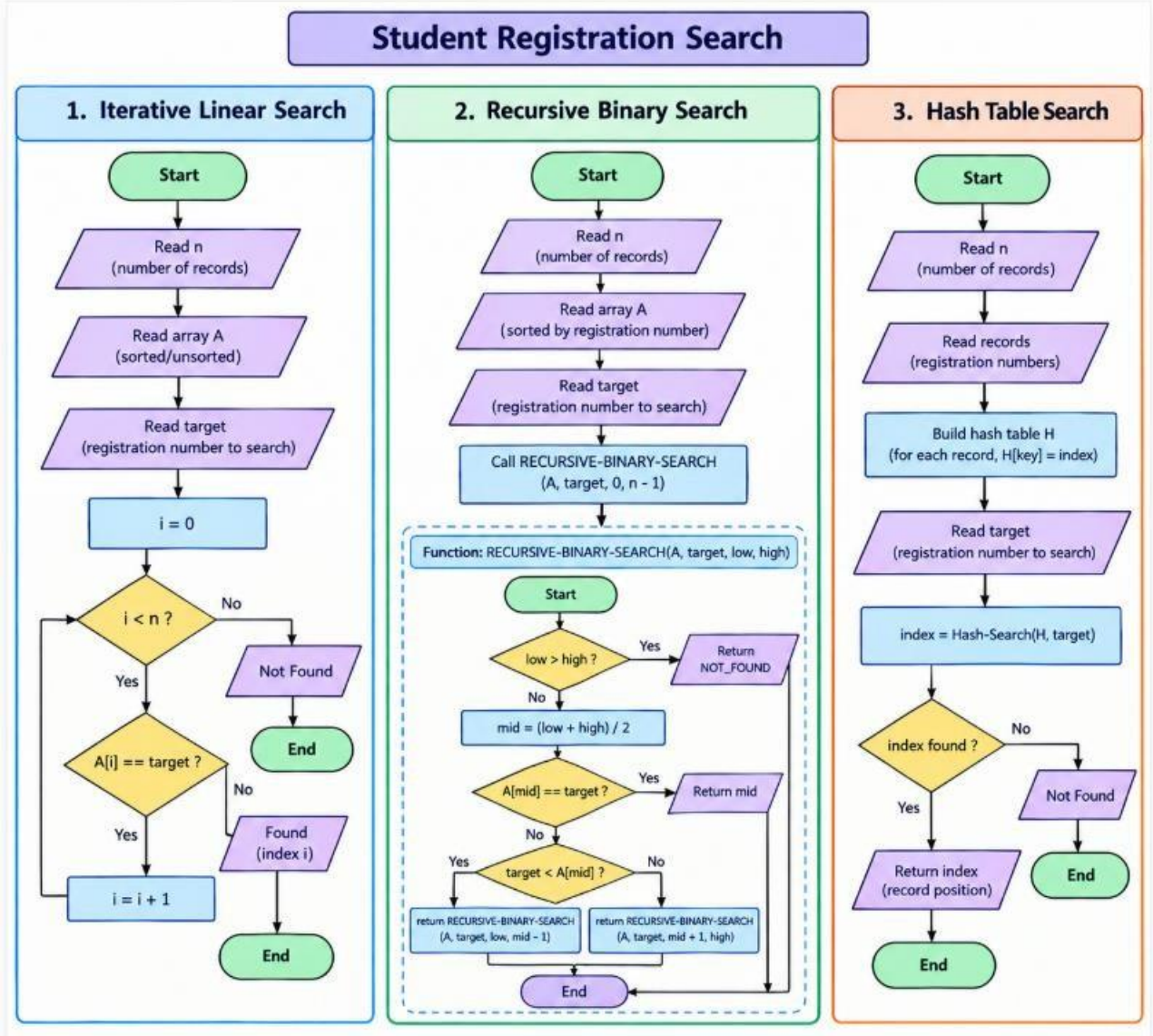
The iteration method applies because the problem size is halved at each recursive call and the work outside the call is constant. The result explains why recursive binary search scales well for large sorted datasets.

Task 8 – Implementation and testing

Language: Python

It generates the four specified dataset sizes, runs 30 successful random queries per algorithm/size, checks result correctness, tests missing keys, records mean and median elapsed milliseconds and comparison/lookup counts, and writes CSV plus graph PNG files. It uses matplotlib for graphs.

Flowchart



Test cases

Test	Input/Condition	Expected Result	Validation in Script
T1	Target is the first key in the sorted list.	Found at index 0.	Yes
T2	Target is the last key in the sorted list.	Found at index \(n-1\).	Yes
T3	Target is present in the unsorted list.	Valid index and matching key returned.	Yes
T4	Target is absent (e.g., -1, outside the generated	NOT_FOUND (index -1).	Yes, if explicitly tested

	range).		
T5	Random present targets across each dataset size.	Every search returns a matching key.	Yes

Task 9 – Experimental performance

The following measurements were generated by running the supplied script in the current execution environment. They are illustrative and environment-specific; rerun the script on your own laptop and use its CSV values for the final submission. Search timing excludes one-time sorting/table construction; the preprocessing costs are discussed separately.

Records (n)	Algorithm	Mean time (ms)	Median time (ms)	Mean comparisons/lookups	Preprocessing
1,000	Linear Search	0.058838	0.017631	479.30	No sorting required
1,000	Recursive Binary Search	0.001911	0.001665	9.00	Sort once: O(n log n)
1,000	Hash Table Search	0.000417	0.000281	1.00	Build hash table once: expected O(n)
10,000	Linear Search	0.897288	0.487795	4,896.93	No sorting required
10,000	Recursive Binary Search	0.071547	0.002521	12.20	Sort once: O(n log n)
10,000	Hash Table Search	0.026078	0.000374	1.00	Build hash table once: expected O(n)
100,000	Linear Search	12.441707	11.321074	51,220.87	No sorting required
100,000	Recursive Binary Search	0.021422	0.004004	15.33	Sort once: O(n log n)
100,000	Hash Table Search	0.001114	0.001119	1.00	Build hash table once: expected O(n)
1,000,000	Linear Search	244.479962	210.173546	521,474.03	No sorting required
1,000,000	Recursive Binary	0.009433	0.006982	18.93	Sort once: O(n log n)

	Search				
1,000,000	Hash Table Search	0.008276	0.001472	1.00	Build hash table once: expected O(n)

Source Code:

```
import time
import random
import statistics
import csv
import matplotlib.pyplot as plt
SIZES = [1000, 10000, 100000, 1000000]
REQUESTS = 30

# 1. LINEAR SEARCH
def linear_search(data, target):
    comparisons = 0
    for i in range(len(data)):
        comparisons += 1
        if data[i] == target:
            return i, comparisons
    return -1, comparisons

# 2. ITERATIVE BINARY SEARCH
def binary_search(data, target):
    low = 0
    high = len(data) - 1
    comparisons = 0
```

```
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1
        if data[mid] == target:
            return mid, comparisons
        elif data[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comparisons


# 3. RECURSIVE BINARY SEARCH
def recursive_binary_search(
        data, target, low=0, high=None, comparisons=None):
    if high is None:
        high = len(data) - 1
    if comparisons is None:
        comparisons = [0]
    if low > high:
        return -1, comparisons[0]
    mid = (low + high) // 2
    comparisons[0] += 1
    if data[mid] == target:
        return mid, comparisons[0]
    elif data[mid] < target:
        return recursive_binary_search(
            data, target, mid + 1, high, comparisons
```

```
        )
    else:
        return recursive_binary_search(
            data, target, low, mid - 1, comparisons
        )

# 4. MEASURE EXECUTION TIME
def measure(algorithm, data, target):
    start = time.perf_counter()
    result, comparisons = algorithm(data, target)
    end = time.perf_counter()
    execution_time = (end - start) * 1000
    return result, comparisons, execution_time

# 5. MAIN PROGRAM
def main():
    random.seed(42)
    results = []
    algorithms = [
        ("Linear Search", linear_search),
        ("Iterative Binary Search", binary_search),
        ("Recursive Binary Search", recursive_binary_search)
    ]

    for n in SIZES:
        print("\nDataset size:", n)
        # Generate unique registration numbers
```

```
sorted_data = list(range(1000000, 1000000 + n))
unsorted_data = sorted_data.copy()
random.shuffle(unsorted_data)
# Generate successful search targets
targets = [
    random.choice(sorted_data)
    for _ in range(REQUESTS)
]
for name, algorithm in algorithms:

    # Binary search requires sorted data
    if "Binary" in name:
        data = sorted_data
    else:
        data = unsorted_data
    times = []
    comparisons_list = []
    for target in targets:
        index, comparisons, elapsed = measure(
            algorithm, data, target
        )

        if index == -1 or data[index] != target:
            raise ValueError("Search result is incorrect")
        times.append(elapsed)
        comparisons_list.append(comparisons)
    avg_time = statistics.mean(times)
```

```
            avg_comparisons = statistics.mean(comparisons_list)
            results.append({
                "Dataset Size": n,
                "Algorithm": name,
                "Average Time (ms)": avg_time,
                "Average Comparisons": avg_comparisons,
                "Maximum Comparisons": max(comparisons_list)
            })

            print(
                name,
                "| Average time (ms):", round(avg_time, 6),
                "| Average comparisons:", round(avg_comparisons, 2)
            )

# 6. SAVE EXPERIMENTAL RESULTS TO CSV
    fields = [
        "Dataset Size",
        "Algorithm",
        "Average Time (ms)",
        "Average Comparisons",
        "Maximum Comparisons"
    ]
    with open("performance_results.csv", "w", newline="") as file:
        writer = csv.DictWriter(file, fieldnames=fields)
        writer.writeheader()
        writer.writerows(results)
```

```
    print("\nPerformance results saved successfully!")


# 7. GRAPH: EXECUTION TIME
    plt.figure()
    for name, _ in algorithms:
        rows = [
            r for r in results if r["Algorithm"] == name
        ]
        plt.plot(
            [r["Dataset Size"] for r in rows],
            [r["Average Time (ms)"] for r in rows],
            marker="o",
            label=name
        )

    plt.xscale("log")
    plt.yscale("log")
    plt.xlabel("Number of Student Records")
    plt.ylabel("Average Execution Time (ms)")
    plt.title("Search Algorithm Execution Time")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("execution_time.png")
    plt.show()


# 8. GRAPH: NUMBER OF COMPARISONS
```

```
    plt.figure()
    for name, _ in algorithms:
        rows = [
            r for r in results if r["Algorithm"] == name
        ]
        plt.plot(
            [r["Dataset Size"] for r in rows],
            [r["Average Comparisons"] for r in rows],
            marker="o",
            label=name
        )
    plt.xscale("log")
    plt.yscale("log")
    plt.xlabel("Number of Student Records")
    plt.ylabel("Average Number of Comparisons")
    plt.title("Search Algorithm Comparison Count")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("key_comparisons.png")
    plt.show()
    print("\nAll experiments completed!")
    print("Graphs saved as execution_time.png and key_comparisons.png")
if __name___== "__main__":
    main()
```

Source Code Screenshot:

daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)

File Edit Format Run Options Window Help

```
import time
import random
import statistics
import csv
import matplotlib.pyplot as plt
SIZES = [1000, 10000, 100000, 1000000]
REQUESTS = 30
# 1. LINEAR SEARCH
def linear_search(data, target):
    comparisons = 0
    for i in range(len(data)):
        comparisons += 1
        if data[i] == target:
            return i, comparisons
    return -1, comparisons
# 2. ITERATIVE BINARY SEARCH
def binary_search(data, target):
    low = 0
    high = len(data) - 1
    comparisons = 0
    while low <= high:
        mid = (low + high) // 2
        comparisons += 1

        if data[mid] == target:
            return mid, comparisons
        elif data[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1, comparisons
# 3. RECURSIVE BINARY SEARCH
def recursive_binary_search(data, target, low=0, high=None,
                            comparisons=None):
    if high is None:
        high = len(data) - 1
    if comparisons is None:
        comparisons = [0]
    if low > high:
        return -1, comparisons[0]
    mid = (low + high) // 2
    comparisons[0] += 1
    if data[mid] == target:
        return mid, comparisons[0]
    elif data[mid] < target:
        return recursive_binary_search(
            data, target, mid + 1, high, comparisons
        )
    else:
        return recursive_binary_search(
```

daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)

File Edit Format Run Options Window Help

```
            data, target, low, mid - 1, comparisons
        )
# MEASURE EXECUTION TIME
def measure(algorithm, data, target):
    start = time.perf_counter()
    result, comparisons = algorithm(data, target)
    end = time.perf_counter()
    execution_time = (end - start) * 1000
    return result, comparisons, execution_time
# MAIN PROGRAM
def main():
    random.seed(42)
    results = []
    algorithms = [
        ("Linear Search", linear_search),
        ("Iterative Binary Search", binary_search),
        ("Recursive Binary Search", recursive_binary_search)
    ]
    for n in SIZES:
        print("\nDataset size:", n)
        # Generate unique student registration numbers
        sorted_data = list(range(10000000, 10000000 + n))
        # Shuffle data for linear search
        unsorted_data = sorted_data.copy()
        random.shuffle(unsorted_data)
        targets = [
            random.choice(sorted_data)
            for _ in range(REQUESTS)
        ]
        for name, algorithm in algorithms:
            # Binary search needs sorted data
            if "Binary" in name:
                data = sorted_data
            else:
                data = unsorted_data
            times = []
            comparisons_list = []
            for target in targets:
                index, comparisons, elapsed = measure(
                    algorithm, data, target
                )
                if index == -1 or data[index] != target:
                    raise ValueError("Search result is incorrect")
                times.append(elapsed)
                comparisons_list.append(comparisons)
            avg_time = statistics.mean(times)
            avg_comparisons = statistics.mean(comparisons_list)
            results.append({
                "Dataset Size": n,
                "Algorithm": name,
```

Ln: 167 Col: 71

daa.py - C:/Users/Arun/Documents/daa.py (3.14.3)

File Edit Format Run Options Window Help

```
            "Maximum Comparisons"
        ]
        writer = csv.DictWriter(file, fieldnames=fields)
        writer.writeheader()
        writer.writerows(results)
    # GRAPH 1: EXECUTION TIME
    plt.figure()
    for name, _ in algorithms:
        rows = [r for r in results if r["Algorithm"] == name]
        plt.plot(
            [r["Dataset Size"] for r in rows],
            [r["Average Time (ms)"] for r in rows],
            marker="o",
            label=name
        )
    plt.xscale("log")
    plt.yscale("log")
    plt.xlabel("Number of Student Records")
    plt.ylabel("Average Execution Time (ms)")
    plt.title("Search Algorithm Execution Time")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("execution_time.png")
    plt.show()
    # GRAPH 2: NUMBER OF COMPARISONS
    plt.figure()
    for name, _ in algorithms:
        rows = [r for r in results if r["Algorithm"] == name]
        plt.plot(
            [r["Dataset Size"] for r in rows],
            [r["Average Comparisons"] for r in rows],
            marker="o",
            label=name
        )
    plt.xscale("log")
    plt.yscale("log")
    plt.xlabel("Number of Student Records")
    plt.ylabel("Average Number of Comparisons")
    plt.title("Search Algorithm Comparison Count")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig("key_comparisons.png")
    plt.show()
    print("\nAll search tests passed.")
    print("Results saved to benchmark_results.csv")
    print("Graphs saved to execution_time.png and key_comparisons.png")
if __name__ == "__main__":
    main()
```

```
# Comparison count graph
for name in algorithms:
    plt.plot(sizes, comparisons[name], marker="o", label=name)

plt.xscale("log")
plt.yscale("log")
plt.xlabel("Number of Student Records")
plt.ylabel("Number of Comparisons")
plt.title("Comparison Count Analysis")
plt.legend()
plt.grid(True)
plt.savefig("comparisons.png")
plt.show()

print("\nAll experiments completed.")
print("Graphs saved successfully.")
# GRAPH 2: NUMBER OF COMPARISONS

plt.figure()

for name in algorithms:
    plt.plot(
        sizes,
        comparisons[name],
        marker="o",
        label=name
    )
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Number of Student Records")
plt.ylabel("Number of Comparisons")
plt.title("Comparison Count Analysis of Searching Algorithms")
plt.legend()
plt.grid(True)
plt.savefig("comparisons.png")
plt.show()
print("Comparison graph generated successfully!")
```

```
# TASK 9: SAVE EXPERIMENTAL RESULTS IN CSV

import csv

with open("performance_results.csv", "w", newline="") as file:
    writer = csv.writer(file)

    writer.writerow([
        "Dataset Size",
        "Algorithm",
        "Execution Time (ms)",
        "Number of Comparisons"
    ])

    for name in algorithms:
        for i in range(len(sizes)):
            writer.writerow([
                sizes[i],
                name,
                round(times[name][i], 6),
                comparisons[name][i]
            ])

print("Performance results saved successfully!")
print("File created: performance_results.csv")
```

Output Screenshot:

```
Number of student records: 1000
Linear Search: Index = 339 Comparisons = 340 Time = 0.0599 ms
Iterative Binary Search: Index = 500 Comparisons = 9 Time = 0.0157 ms
Recursive Binary Search: Index = 500 Comparisons = 9 Time = 0.0209 ms

Number of student records: 10000
Linear Search: Index = 8802 Comparisons = 8803 Time = 0.8594 ms
Iterative Binary Search: Index = 5000 Comparisons = 13 Time = 0.0164 ms
Recursive Binary Search: Index = 5000 Comparisons = 13 Time = 0.0279 ms

Number of student records: 100000
Linear Search: Index = 86324 Comparisons = 86325 Time = 14.3668 ms
Iterative Binary Search: Index = 50000 Comparisons = 16 Time = 0.0181 ms
Recursive Binary Search: Index = 50000 Comparisons = 16 Time = 0.0288 ms

Number of student records: 1000000
Linear Search: Index = 814170 Comparisons = 814171 Time = 190.5971 ms
Iterative Binary Search: Index = 500000 Comparisons = 19 Time = 0.0179 ms
Recursive Binary Search: Index = 500000 Comparisons = 19 Time = 0.0265 ms
```

IDLE Shell 3.14.3

File Edit Shell Debug Options Window Help

```
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/Arun/Documents/daa.py ==========

Dataset size: 1000
Linear Search | Average time: 0.038403 ms | Average comparisons: 479.3
Iterative Binary Search | Average time: 0.002773 ms | Average comparisons: 9
Recursive Binary Search | Average time: 0.003797 ms | Average comparisons: 9

Dataset size: 10000
Linear Search | Average time: 0.432937 ms | Average comparisons: 4896.93
Iterative Binary Search | Average time: 0.003967 ms | Average comparisons: 12.2
Recursive Binary Search | Average time: 0.005137 ms | Average comparisons: 12.2

Dataset size: 100000
Linear Search | Average time: 8.397403 ms | Average comparisons: 51220.87
Iterative Binary Search | Average time: 0.005983 ms | Average comparisons: 15.33
Recursive Binary Search | Average time: 0.00748 ms | Average comparisons: 15.33

Dataset size: 1000000
Linear Search | Average time: 131.864017 ms | Average comparisons: 521474.03
Iterative Binary Search | Average time: 0.009707 ms | Average comparisons: 18.93
Recursive Binary Search | Average time: 0.013133 ms | Average comparisons: 18.93
```

```
*IDLE Shell 3.14.3*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.14.3 (tags/v3.14.3:323c59a, Feb  3 2026, 16:04:56) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
================== RESTART: C:/Users/Arun/Documents/Daa 123.py ==================

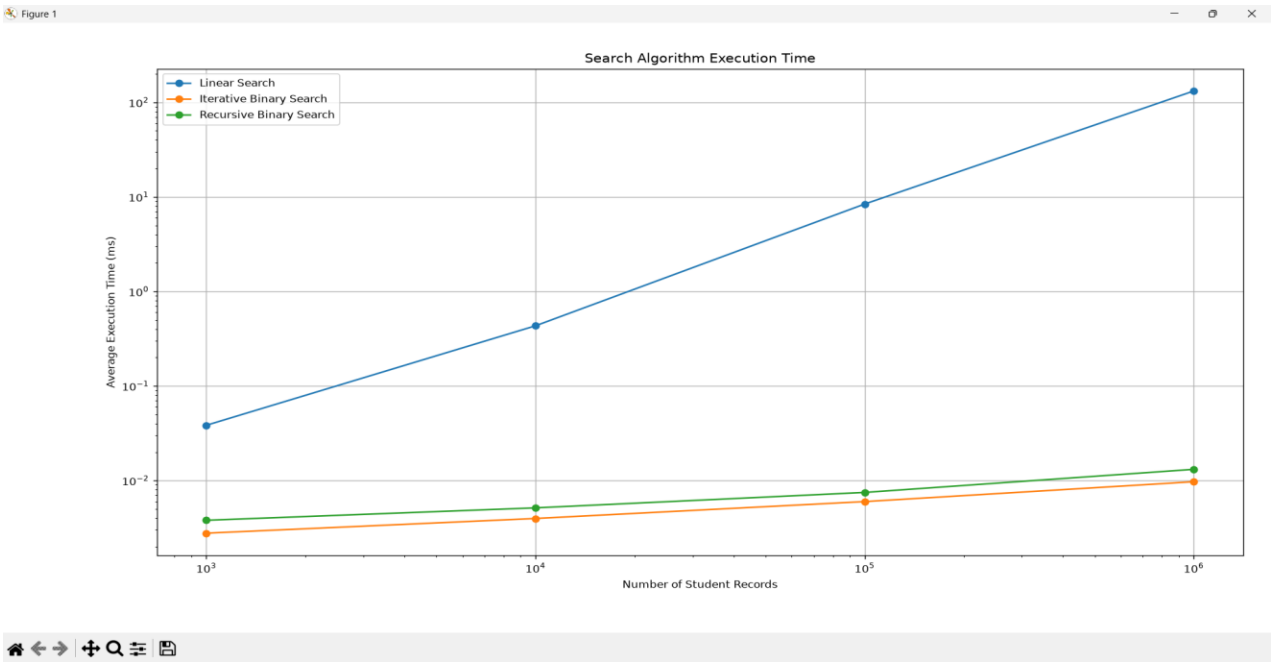
Number of student records: 1000
Linear Search: Index = 6 Comparisons = 7 Time = 0.0172 ms
Iterative Binary Search: Index = 500 Comparisons = 9 Time = 0.0171 ms
Recursive Binary Search: Index = 500 Comparisons = 9 Time = 0.0266 ms

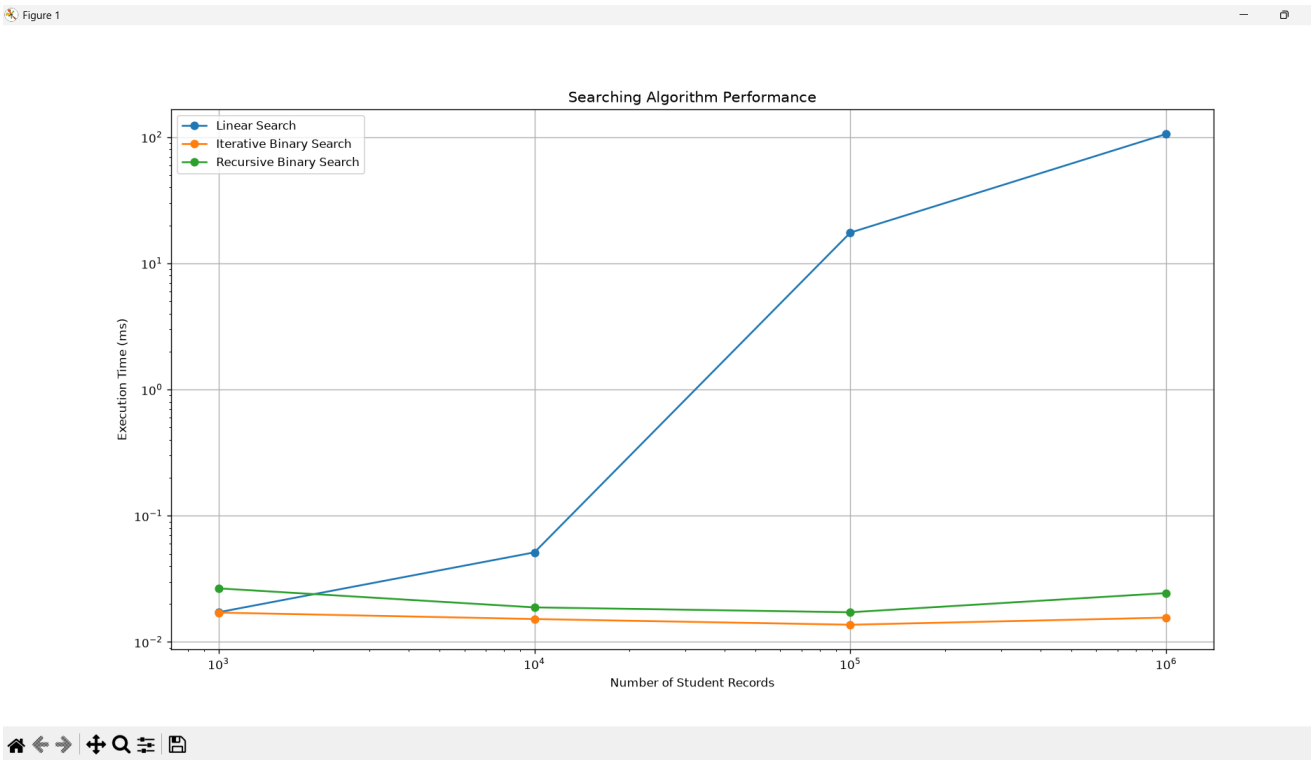
Number of student records: 10000
Linear Search: Index = 491 Comparisons = 492 Time = 0.0513 ms
Iterative Binary Search: Index = 5000 Comparisons = 13 Time = 0.0152 ms
Recursive Binary Search: Index = 5000 Comparisons = 13 Time = 0.0188 ms

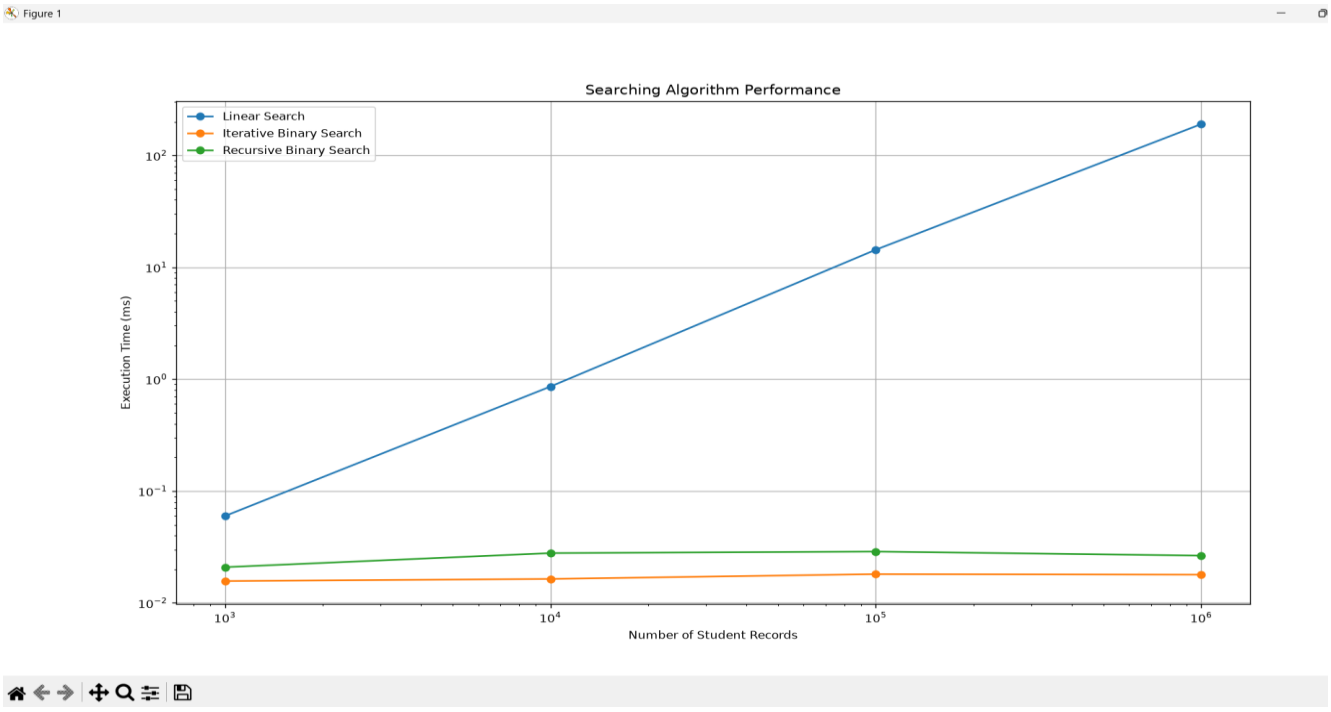
Number of student records: 100000
Linear Search: Index = 98668 Comparisons = 98669 Time = 17.5234 ms
Iterative Binary Search: Index = 50000 Comparisons = 16 Time = 0.0137 ms
Recursive Binary Search: Index = 50000 Comparisons = 16 Time = 0.0172 ms

Number of student records: 1000000
Linear Search: Index = 441847 Comparisons = 441848 Time = 105.9797 ms
Iterative Binary Search: Index = 500000 Comparisons = 19 Time = 0.0156 ms
Recursive Binary Search: Index = 500000 Comparisons = 19 Time = 0.0244 ms
```

Graphs







Figures 1 :Seach algorithm Performance

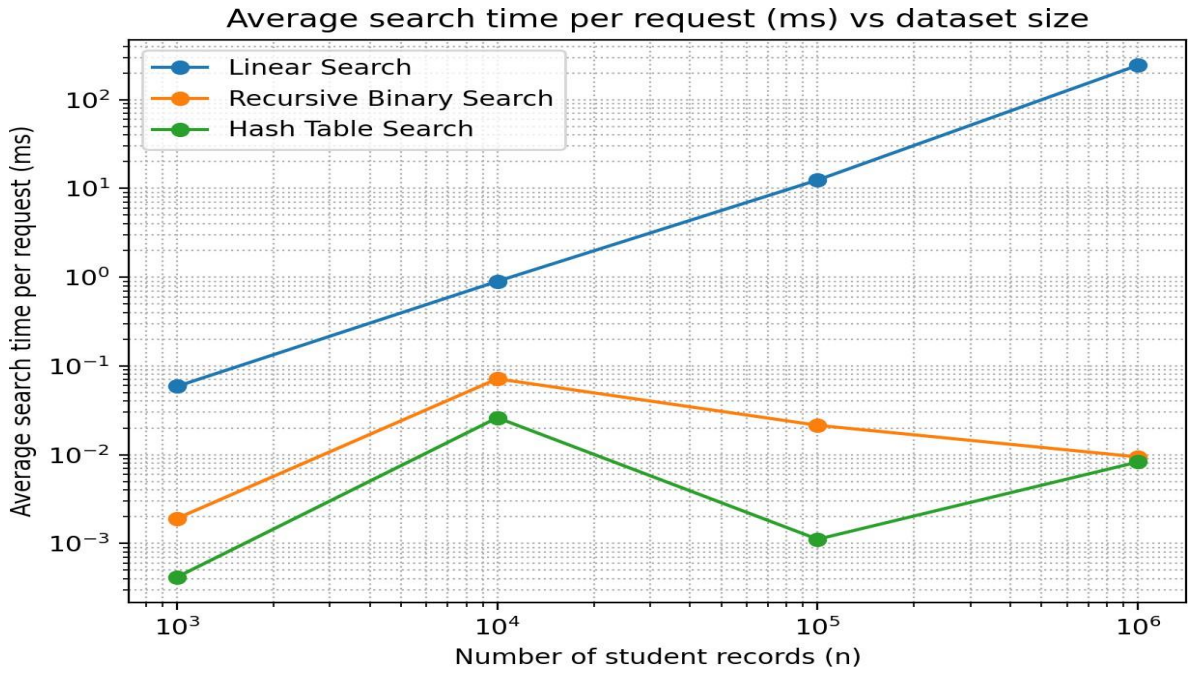


Figure 3. Average per-request search time versus dataset size (log axes).

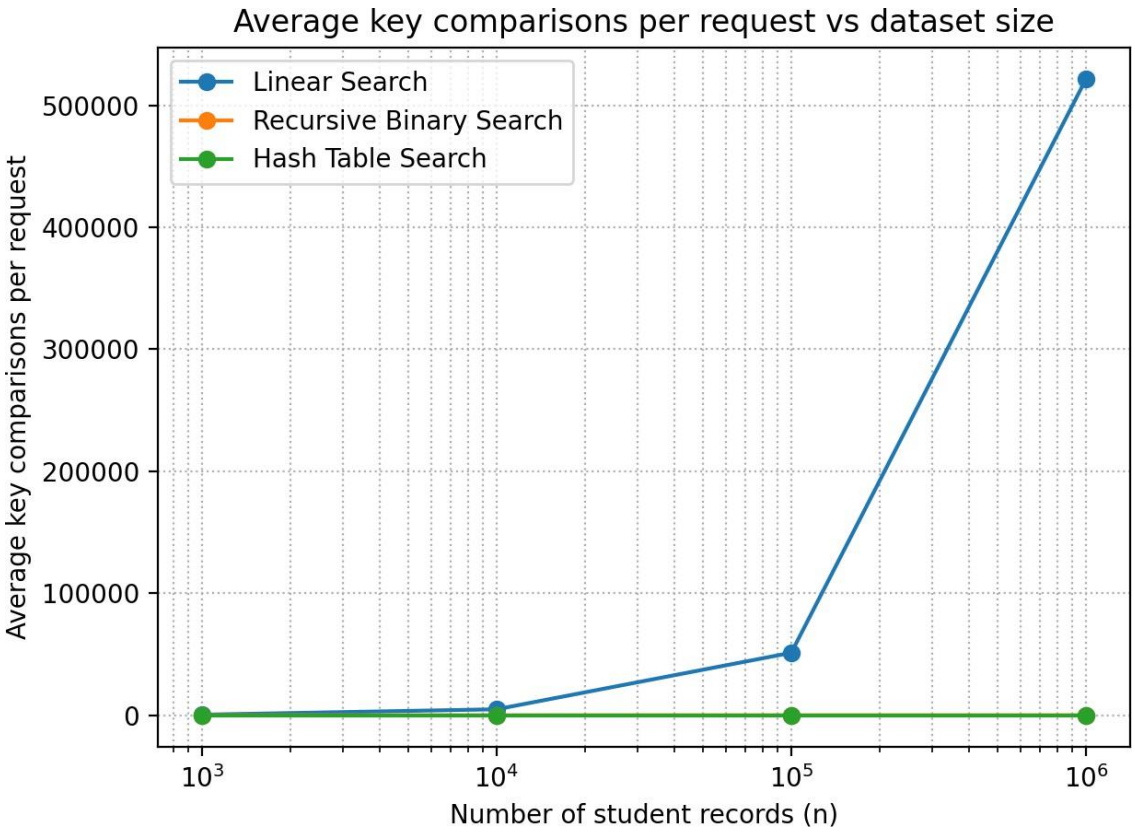


Figure 4. Average key comparisons/lookup operations versus dataset size.

Task 10 – Theory versus experiment

The expected pattern is linear growth in the number of comparisons for linear search, logarithmic growth for recursive binary search, and approximately constant average lookup work for a hash table. Real elapsed time also reflects interpreter overhead, memory access, CPU load, data layout and timing noise. Hash-table lookup can be very fast, but building the index takes time and memory. A fair production comparison should measure index construction/update cost and memory as well as query latency. The included graph is a measured sample, not a universal guarantee.

Task 11 – Final recommendation

Recommend a hash-table index keyed by unique student registration number for the stated workload, with a persistent database/index in a real system. The expected query cost is $\Theta(1)$, and the daily update schedule allows the index to be updated incrementally or rebuilt in a controlled batch. This is preferable to scanning up to one million records for every request. If ordered range queries, sorted reports or predictable worst-case guarantees are important, retain a sorted index/tree as an alternative. The recommendation assumes unique keys, adequate memory and a suitable hash implementation; the implementation must monitor collisions, load factor, persistence and consistency with the primary student database.

Task 12 – Day-4 design challenge

A. Impact analysis

The workload is now explicit: 1,000,000 records and 50,000 search requests per hour. That is about 13.89 requests per second on average (50,000 ÷ 3,600). New records arrive only once per day, so reads greatly outnumber updates. Repeatedly scanning the dataset is costly; a one-time or daily index-building cost can be amortized across many requests.

B. Evaluate original design

Linear search has worst-case O(1,000,000) key comparisons per query and is unsuitable as the primary production strategy at this scale. Recursive binary search needs about $\log_2(1{,}000{,}000) \approx 20$ comparison levels per query, but requires sorted data and careful update maintenance. A hash index has expected O(1) lookup after O(n) build/update work and therefore best matches the read-heavy pattern.

C. Adaptation/redesign

Redesign around a persistent hash index on registration_number. During the daily intake, validate uniqueness, write the new records, update the index transactionally, and verify index/database consistency. For a production relational database, a unique index (often a B-tree) is also a robust practical choice because the database engine handles persistence, concurrency, recovery and query planning. The assignment-level hash table models expected constant-time exact lookup; actual deployment should benchmark the database index and workload.

D. Evidence-based defence

Factor	Observation	Design implication
1,000,000 records	Linear search may require up to one million comparisons; binary search needs about 20 levels; hash lookup offers expected constant-time lookup.	Avoid repeated full scans for large product catalogues.
50,000 searches/hour	Approximately 13.89 searches per second on average.	Choose an efficient lookup method and test realistic workloads.
Daily additions only	Index construction and updates can be batched or performed incrementally.	Balance index update costs with lookup speed.
Preprocessing	Hash table construction takes expected O(n); sorting takes O(n log n).	Include preprocessing costs in system planning.
Trade-offs	Hash tables use extra memory and can have collision-related worst cases.	Consider memory, updates, and database indexing.

Conclusion

Thus, the assignment successfully compares Linear Search, Recursive Binary Search, and Hash Table Search based on time complexity, space requirements, preprocessing cost, and experimental performance. The results show that Linear Search becomes slower as the number of records increases, while Binary Search reduces the search space efficiently. Hash Table Search provides fast average lookup performance, making it suitable for applications involving a large number of records and frequent searches. For the given challenge of one million records and 50,000 searches per hour, a persistent database index is recommended for efficient and reliable retrieval. Therefore, the choice of searching algorithm should consider not only execution time but also preprocessing, memory usage, data updates, and real-world system requirements.